University of Central Missouri

Department of Computer Science & Cybersecurity

CS5760 Natural Language Processing

Spring 2026

Homework 4.

Student name: CHINTALA SAI SRAVAN

Submission Requirements:

- Once finished your assignment push your source code to your repo (GitHub) and explain the work through the ReadMe file properly. Make sure you add your student info in the ReadMe file.
- Submit your GitHub link on the Bright Space.
- Comment your code appropriately ***IMPORTANT.***
- Any submission after provided deadline is considered as a late submission.

Q6. Multi-Input Feedforward

Consider a neural network with 3 inputs x1=2, x2=1, x3=3, one hidden layer of 2 sigmoid units, and one sigmoid output.

Weights:

$$A = \begin{bmatrix} 0.2 & -0.5 \\ 0.1 & 0.3 \\ -0.2 & 0.8 \\ 0.4 & -0.6 \end{bmatrix}, \quad B = \begin{bmatrix} 0.7 \\ -1.2 \\ 0.5 \end{bmatrix}$$

- A includes biases (last row).
- B includes bias (last element).

Tasks

a) Compute hidden pre-activations and activations.

Solution: Given: $x_1 = 2,\ x_2 = 1,\ x_3 = 3$

Add Bias: $x = [2,1,3,1]$

Compute $z_1 = (2)(0.2) + (1)(0.1) + (3)(-0.2) + (1)(0.4)$

$= 0.4 + 0.1 - 0.6 + 0.4 = 0.3$

Compute $z_2 = (2)(-0.5) + (1)(0.3) + (3)(0.8) + (1)(-0.6)$

$= -1 + 0.3 + 2.4 - 0.6 = 1.1$

Apply Sigmoid

$$h_1 = \sigma(0.3) = \frac{1}{1 + e^{-0.3}}$$

$$e^{-0.3} \approx 0.741$$

$$h_1 \approx \frac{1}{1.741} \approx 0.574$$

$$h_2 = \sigma(1.1) = \frac{1}{1 + e^{-1.1}}$$

$$e^{-1.1} \approx 0.333$$

$$h_2 \approx \frac{1}{1.333} \approx 0.750$$

b) Compute the output activation y.

Solution: Add bias: $h = [0.574,\ 0.750,\ 1]$

Compute output $z = (0.574)(0.7) + (0.750)(-1.2) + (1)(0.5)$

$$z = 0.402 - 0.900 + 0.5 = 0.002$$

Final Output (Sigmoid): $y = \sigma(0.002)$

$$e^{-0.002} \approx 0.998$$

$$y \approx \frac{1}{1.998} \approx 0.5005$$

c) If the true label is $t = 1$, calculate the binary cross-entropy loss.

Solution: Loss (Binary Cross-Entropy): $t = 1$

$$L = -\log(y)$$

$$L = -\log(0.5005)$$

$$L \approx 0.692$$

Q7. XOR with ReLU Network

We use the XOR ReLU network from the slides

$$h_1 = \mathrm{ReLU}(x_1 + x_2), \quad h_2 = \mathrm{ReLU}(x_1 + x_2 - 1), \quad y = \mathrm{ReLU}(h_1 - 2h_2)$$

Extend it with an additional hidden unit:

$$h_3 = \mathrm{ReLU}(2x_1 - x_2)$$

and new output:

$$y = \mathrm{ReLU}(h_1 - 2h_2 + h_3)$$

Tasks

a) Compute outputs for all four XOR inputs (0,0),(0,1),(1,0),(1,1).

Solution:

Case 1: (0, 0)

$$h_1 = \mathrm{ReLU}(0 + 0) = 0$$
$$h_2 = \mathrm{ReLU}(0 + 0 - 1) = \mathrm{ReLU}(-1) = 0$$
$$h_3 = \mathrm{ReLU}(0 - 0) = 0$$
$$y = \mathrm{ReLU}(0 - 0 + 0) = 0$$

Case 2: (0, 1)

$$h_1 = \mathrm{ReLU}(1) = 1$$
$$h_2 = \mathrm{ReLU}(0) = 0$$
$$h_3 = \mathrm{ReLU}(-1) = 0$$
$$y = \mathrm{ReLU}(1 - 0 + 0) = 1$$

Case 3: (1, 0)

$$h_1 = \mathrm{ReLU}(1) = 1$$
$$h_2 = \mathrm{ReLU}(0) = 0$$
$$h_3 = \mathrm{ReLU}(2) = 2$$
$$y = \mathrm{ReLU}(1 - 0 + 2) = 3$$

Case 4: (1, 1)

$$h_1 = \mathrm{ReLU}(2) = 2$$
$$h_2 = \mathrm{ReLU}(1) = 1$$
$$h_3 = \mathrm{ReLU}(1) = 1$$
$$y = \mathrm{ReLU}(2 - 2 + 1) = 1$$

Final Outputs Table

Input	**Output**
(0,0)	0
(0,1)	1
(1,0)	3
(1,1)	1

b) Compare the decision boundary with the original 2-hidden-unit XOR network.

Solution:

Original 2-hidden-unit XOR:

- Symmetric in x_1, x_2
- Correctly separates:
  - (0,0), (1,1) → 0
  - (0,1), (1,0) → 1

With Added $h_3 = \text{ReLU}(2x_1 - x_2)$:

- Introduces **asymmetry**
- Gives **extra weight to** x_1
- Shifts decision boundary toward inputs where $x_1 = 1$

Effect:

- Output becomes **biased toward** x_1
- Boundary is no longer symmetric

c) Does this extension still compute XOR exactly? If not, which inputs differ?

Solution:

Expected XOR:

Input	XOR
(0,0)	0
(0,1)	1
(1,0)	1
(1,1)	0

Model Output:

Input	Output	Correct?
(0,0)	0	✅
(0,1)	1	✅
(1,0)	3 → (positive)	✅ (still class 1)
(1,1)	1	❌

- The network **does NOT compute XOR exactly**
- **Incorrect input: (1,1)** → outputs **1 instead of 0**

Q8. Decision Boundary and Misclassification

You train a perceptron with decision rule:

$$y = \begin{cases} 1 & \text{if } w_1x_1 + w_2x_2 + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

with parameters w1=1, w2=−2, b=1.

Dataset:
(2,1)→1, (1,3)→0, (3,2)→1, (0,1)→0.

Tasks
a) Write the decision boundary equation and sketch it.
Solution:
Given: $w_1 = 1,\ w_2 = -2,\ b = 1$
Decision Boundary: $w_1x_1 + w_2x_2 + b = 0$
$$x_1 - 2x_2 + 1 = 0$$
b) Classify each point using the perceptron rule. Which ones are misclassified?
Solution:
Use: $y = \text{sign}(w \cdot x + b)$
Compute each:
(2,1): $2 - 2 + 1 = 1 \to 1$
(1,3): $1 - 6 + 1 = -4 \to 0$
(3,2): $3 - 4 + 1 = 0 \to 0$
(0,1): $0 - 2 + 1 = -1 \to 0$
Misclassification: **(3,2)**
c) Compute the perceptron loss (number of mistakes).
Solution: Loss=#misclassified points = **1**
d) If we apply one weight update with learning rate η=0.5 on the first mistake, what are the new weights?
Solution: Use perceptron update rule: $w = w + \eta \cdot t \cdot x$
$$b = b + \eta \cdot t$$
Given: $\eta = 1$
Update using misclassified point (3,2): $x = (3,2), t = 1$
New weights: $w_1 = 1 + (1)(3) = 4$
$$w_2 = -2 + (1)(2) = 0$$
New bias: $b = 1 + 1 = 2$

Part I. Short Answer

Q3. Neural Networks: Structure, Activation, and Representation
(a) Why are non-linear activation functions (like ReLU, sigmoid, tanh) necessary in neural networks?
Answer: Non-linear activations allow learning **complex relationships**.
Without them → network = linear model.
(b) Why are deep networks better at feature learning compared to shallow models like logistic regression?
Answer: Deep networks:
- learn hierarchical features
- capture complex patterns

1) RNN Families & Use-Cases (Many-to-X)
a) Map each task to the most suitable RNN I/O pattern and explain why (1–2 lines each):
• next-word prediction

• sentiment of a sentence
• NER
• machine translation
(Choose from: one-to-many, many-to-one, many-to-many aligned, many-to-many unaligned.)
Answer:

Task	Type
Next-word prediction	One-to-many
Sentiment	Many-to-one
NER	Many-to-many (aligned)
Translation	Many-to-many (unaligned)

b) In one sentence, explain how "unrolling" over time enables BPTT and weight sharing.
Answer: Unrolling a recurrent network over time converts it into a sequence of identical layers with shared parameters, allowing **backpropagation through time (BPTT)** to compute gradients across all time steps while using the **same weights repeatedly**.

c) Give one advantage and one limitation of weight sharing across time in RNNs.
Answer:

- Advantage: Fewer parameters
- Limitation: Cannot capture position-specific behaviour

2) Vanishing Gradients & Remedies
a) Describe the vanishing gradient problem in RNNs and how it affects long-range dependencies.
Answer: The **vanishing gradient problem** occurs when gradients become extremely small as they are backpropagated through many time steps in an RNN.
Because of repeated multiplication by small values (from derivatives like sigmoid/tanh), the gradients shrink exponentially.
Effect:

- Early time steps receive almost **no learning signal**
- The model fails to learn **long-range dependencies**
- It focuses only on **recent inputs**, ignoring earlier context

b) List two *architectural* solutions and briefly how each helps gradient flow.
Answer:
1. LSTM (Long Short-Term Memory)

- Uses **gates (input, forget, output)**
- Maintains a **cell state with linear connections**
- Allows gradients to flow without vanishing

Helps preserve long-term information
2. GRU (Gated Recurrent Unit)

- Simplified version of LSTM with **update and reset gates**
- Controls how much past information is retained

Reduces vanishing gradients while being computationally efficient

c) Give one *training* technique (not an architecture change) that can mitigate the issue and why.

Answer:
Gradient Clipping

- Limits the maximum value of gradients during training
- Prevents gradients from becoming too large or unstable

Helps stabilize training and indirectly reduces issues related to vanishing/exploding gradients

3) LSTM Gates & Cell State

a) Explain the roles of the forget, input, and output gates. For each, name its activation and purpose.

Answer:

1. Forget Gate

- **Activation:** Sigmoid
- **Purpose:** Decides what information to **remove** from the cell state
- **Role:** Helps discard irrelevant past information
- Output values between 0 and 1:
  - 0 → completely forget
  - 1 → completely keep

2. Input Gate

- **Activation:** Sigmoid (gate) + Tanh (candidate values)
- **Purpose:** Decides what **new information to add** to the cell state
- **Role:** Controls how much new information enters memory

3. Output Gate

- **Activation:** Sigmoid (gate) + Tanh (cell state)
- **Purpose:** Decides what part of the cell state to **output as hidden state**
- **Role:** Controls what information is exposed to the next layer/time step

b) Why is the LSTM cell state often described as providing a "linear path" for gradients?

Answer: The LSTM cell state is considered a **linear path** because information flows through it with only **minor linear interactions (addition and multiplication)** and no repeated nonlinear transformations.

This allows gradients to:

- flow **unchanged over many time steps**
- avoid shrinking exponentially

This is why LSTMs can capture **long-term dependencies**

c) In one or two sentences, contrast "what to remember" vs. "what to expose" in LSTMs.

Answer:

- **What to remember:** Controlled by the **forget and input gates**, which decide what information stays in the cell state.
- **What to expose:** Controlled by the **output gate**, which decides what part of that stored information is passed forward as the hidden state.

4) Self-Attention

a) Define Query (Q), Key (K), and Value (V) in the context of self-attention.

Answer: In self-attention, each input token is transformed into three vectors:

- **Query (Q):** Represents what the current token is **looking for**

- **Key (K):** Represents what each token **offers/matches against**
- **Value (V):** Represents the **actual information/content** of the token

Attention is computed by comparing **Q with K** to decide how much of **V** to use.

b) Write the formula for dot-product attention.

Answer: $\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$

c) Why do we divide by $\sqrt{d_k}$?

Answer: We divide by $\sqrt{d_k}$to **scale down large dot-product values**.

Without scaling:

- Large values → **softmax becomes very sharp**
- Leads to **unstable gradients**

Scaling ensures:

- stable training
- smoother attention distribution

5) Multi-Head Attention & Residual Connections

a) Why do Transformers use multi-head attention instead of single-head attention?

Answer: Transformers use **multi-head attention** instead of a single head to allow the model to **learn multiple relationships in parallel**.

Each head:

- attends to **different parts of the sequence**
- captures **different types of patterns**

Benefit:

- richer representations
- better understanding of context

A single head would focus on only one type of relationship, limiting performance.

b) What is the purpose of Add & Norm (Residual + LayerNorm)? Explain two benefits.

Answer: Add (Residual Connection): Adds input of the layer to its output: Output $= x +$ Sublayer(x)

LayerNorm: Normalizes outputs across features

Two Benefits:

1. **Improved Gradient Flow**
    - Residual connections help gradients pass through deep networks
    - Prevents vanishing gradient problem
2. **Stable & Faster Training**
    - LayerNorm stabilizes activations
    - Makes training more consistent and faster

c) Describe one example of linguistic relation that different heads might capture (e.g., coreference, syntax).

Answer: One attention head might capture **coreference**:

Example: "John went to the store because **he** needed milk."

- The model learns that **"he" refers to "John"**

Other possible relations:

- syntax (subject–verb agreement)
- semantic similarity
- positional relationships

6) Encoder–Decoder with Masked Attention

a) Why does the decoder use masked self-attention? What problem does it prevent?

Answer: The decoder uses **masked self-attention** to ensure that each position can only attend to **previous tokens, not future ones**.

Problem it prevents:

- Prevents the model from **"cheating"** by seeing future words during training
- Maintains the **causal (left-to-right) generation** property

This ensures the model learns to predict the next word based only on past context.

b) What is the difference between encoder self-attention and encoder–decoder cross-attention?

Answer:

Encoder Self-Attention

- Each token attends to **all other tokens in the input sequence**
- Uses: $Q = K = V =$ encoder inputs

Purpose: build contextual representation of input

Encoder–Decoder Cross-Attention

- Decoder attends to **encoder outputs**
- Uses: $Q =$ decoder, $K, V =$ encoder outputs

Purpose: helps decoder **focus on relevant input parts while generating output**

c) During inference (no teacher forcing), how does the model generate tokens step by step?

Answer: During inference, the model generates tokens **one step at a time**:

1. Start with a special **start token (<SOS>)**
2. Generate the first output token
3. Feed the generated token back into the decoder
4. Predict the next token
5. Repeat until **<EOS> (end token)** is produced

This is called **autoregressive generation**

Part II: Programming

Q1. Character-Level RNN Language Model ("hello" toy & beyond)

Goal: Train a tiny character-level RNN to predict the next character given previous characters.

Data (toy to start):

- Start with a small toy corpus you create (e.g., several "hello…", "help…", short words/sentences).
- Then expand to a short plain-text file of ~50–200 KB (any public-domain text of your choice).

Model:

- Embedding → RNN (Vanilla RNN or GRU or LSTM) → Linear → Softmax over characters.
- Hidden size 64–256; sequence length 50–100; batch size 64; train 5–20 epochs.

Train:

- Teacher forcing (use the true previous char as input during training).
- Cross-entropy loss; Adam optimizer.

Report:

1. Training/validation loss curves.
2. Sample 3 temperature-controlled generations (e.g., $\tau = 0.7, 1.0, 1.2$) for 200–400 chars each.
3. A 3–5 sentence reflection: what changes when you vary sequence length, hidden size, and temperature?
   (Connect to slides: embedding, sampling loop, teacher forcing, tradeoffs

Code:

```
# char_rnn.py
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import random
import matplotlib.pyplot as plt

# -------------------------
# 1. Load text corpus
# -------------------------
# Replace 'sample.txt' with any public-domain text (~50–200 KB)
with open('sample.txt', 'r', encoding='utf-8') as f:
    text = f.read().lower()  # lowercase for simplicity

# Remove unwanted characters (optional)
text = ''.join(c for c in text if c.isprintable())

# Vocabulary and mappings
chars = sorted(list(set(text)))
```

```python
vocab_size = len(chars)
char2idx = {ch: i for i, ch in enumerate(chars)}
idx2char = {i: ch for i, ch in enumerate(chars)}

# Encode full text
encoded_text = np.array([char2idx[c] for c in text])
print(f"Corpus length: {len(text)}, Vocab size: {vocab_size}")

# -------------------------
# 2. Create sequences
# -------------------------
seq_length = 50
step = 1
inputs, targets = [], []
for i in range(0, len(encoded_text) - seq_length, step):
    inputs.append(encoded_text[i:i+seq_length])
    targets.append(encoded_text[i+seq_length])

inputs = torch.tensor(np.array(inputs), dtype=torch.long)
targets = torch.tensor(np.array(targets), dtype=torch.long)
print("Input shape:", inputs.shape, "Target shape:", targets.shape)

# -------------------------
# 3. Define model
# -------------------------
class CharRNN(nn.Module):
    def __init__(self, vocab_size, embed_size, hidden_size):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_size)
        self.rnn = nn.LSTM(embed_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x, hidden=None):
        x = self.embed(x)
        out, hidden = self.rnn(x, hidden)
        out = self.fc(out[:, -1, :])  # last time step
        return out, hidden
```

```
# ------------------------
# 4. Hyperparameters
# ------------------------
embed_size = 128
hidden_size = 256
batch_size = 64
epochs = 10
lr = 0.002

model = CharRNN(vocab_size, embed_size, hidden_size)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=lr)

# ------------------------
# 5. Train / Validation split
# ------------------------
dataset = list(zip(inputs, targets))
random.shuffle(dataset)
split_idx = int(len(dataset) * 0.9)
train_data = dataset[:split_idx]
val_data = dataset[split_idx:]

train_losses = []
val_losses = []

# ------------------------
# 6. Training loop
# ------------------------
for epoch in range(1, epochs+1):
    # Training
    random.shuffle(train_data)
    total_loss = 0
    for i in range(0, len(train_data), batch_size):
        batch = train_data[i:i+batch_size]
        x_batch = torch.stack([b[0] for b in batch])
        y_batch = torch.tensor([b[1] for b in batch], dtype=torch.long)
```

```
        optimizer.zero_grad()
        out, _ = model(x_batch)
        loss = criterion(out, y_batch)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    avg_train_loss = total_loss / len(train_data)
    train_losses.append(avg_train_loss)

    # Validation
    with torch.no_grad():
        total_val_loss = 0
        for i in range(0, len(val_data), batch_size):
            batch = val_data[i:i+batch_size]
            x_batch = torch.stack([b[0] for b in batch])
            y_batch = torch.tensor([b[1] for b in batch], dtype=torch.long)
            out, _ = model(x_batch)
            loss = criterion(out, y_batch)
            total_val_loss += loss.item()
        avg_val_loss = total_val_loss / len(val_data)
        val_losses.append(avg_val_loss)

    print(f"Epoch {epoch}/{epochs} | Train Loss: {avg_train_loss:.4f} | Val Loss: {avg_val_loss:.4f}")

# -------------------------
# 7. Plot loss curves
# -------------------------
plt.plot(train_losses, label='Train Loss')
plt.plot(val_losses, label='Validation Loss')
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.title("Training and Validation Loss")
plt.show()

# -------------------------
```

```python
# 8. Text generation
# ------------------------
def generate_text(model, start_seq, length=200, temperature=1.0):
    model.eval()
    generated = start_seq
    input_seq = torch.tensor([char2idx[c] for c in start_seq], dtype=torch.long).unsqueeze(0)
    hidden = None
    for _ in range(length):
        out, hidden = model(input_seq, hidden)
        probs = torch.softmax(out / temperature, dim=-1).detach().numpy().flatten()
        char_idx = np.random.choice(len(probs), p=probs)
        generated += idx2char[char_idx]
        input_seq = torch.tensor([[char_idx]], dtype=torch.long)
    return generated

# Generate samples with different temperatures
start_seq = "the "
print("\n=== Temperature 0.7 ===\n", generate_text(model, start_seq, 200, temperature=0.7))
print("\n=== Temperature 1.0 ===\n", generate_text(model, start_seq, 200, temperature=1.0))
print("\n=== Temperature 1.2 ===\n", generate_text(model, start_seq, 200, temperature=1.2))
```

Output:

```
Corpus length: 141312, Vocab size: 49
/tmp/ipykernel_13703/1862751307.py:39: UserWarning: Creating a tensor from a
list of numpy.ndarrays is extremely slow. Please consider converting the list
to a single numpy.ndarray with numpy.array() before converting to a tensor.
(Triggered internally at /pytorch/torch/csrc/utils/tensor_new.cpp:253.)
  inputs = torch.tensor(inputs, dtype=torch.long)
Input shape: torch.Size([141262, 50]) Target shape: torch.Size([141262])
Epoch 1/10 | Train Loss: 0.0286 | Val Loss: 0.0248
Epoch 2/10 | Train Loss: 0.0235 | Val Loss: 0.0233
Epoch 3/10 | Train Loss: 0.0217 | Val Loss: 0.0225
Epoch 4/10 | Train Loss: 0.0204 | Val Loss: 0.0220
Epoch 5/10 | Train Loss: 0.0195 | Val Loss: 0.0218
Epoch 6/10 | Train Loss: 0.0187 | Val Loss: 0.0219
Epoch 7/10 | Train Loss: 0.0181 | Val Loss: 0.0219
Epoch 8/10 | Train Loss: 0.0176 | Val Loss: 0.0219
Epoch 9/10 | Train Loss: 0.0171 | Val Loss: 0.0219
Epoch 10/10 | Train Loss: 0.0167 | Val Loss: 0.0222
```

```
=== Temperature 0.7 ===
 the game, and i went tone, andrepeatedin a tone of the otherworrily. "we
must begrand their hast off reallydogreat dear! no you got things!" alice
said to alice again at all a little girl abshing and

=== Temperature 1.0 ===
 the queer-feet, and book, and thought, but alice who werpents, "if i don't—
just afraniwis her _strearly," the only in a morcook. and alice has more a
long of them, and added and athand make that the pool

=== Temperature 1.2 ===
 the long,_so such a greathereyalred a body thing mann! triatehise, (unfor
yourseps back toahers and on!why did you, wilish i day-of justtable."crong,
and she very know she said to creatures to a vicular t
```

Q2. Mini Transformer Encoder for Sentences

Task: Build a mini Transformer Encoder (NOT full decoder) to process a batch of sentences.

Steps:

1. Use a small dataset (e.g., 10 short sentences of your choice).
2. Tokenize and embed the text.
3. Add sinusoidal positional encoding.
4. Implement:
    - Self-attention layer
    - Multi-head attention (2 or 4 heads)
    - Feed-forward layer
    - Add & Norm
5. Show:
    - Input tokens
    - Final contextual embeddings

- Attention heatmap between words (visual or printed)

Code:

```
# mini_transformer_encoder.py

import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt

# -------------------------
# 1. Small Dataset (10 sentences)
# -------------------------
sentences = [
    "i love machine learning",
    "transformers are powerful models",
    "deep learning is amazing",
    "natural language processing is fun",
    "attention helps models focus",
    "pytorch makes training easy",
    "i enjoy learning new things",
    "data science is interesting",
    "neural networks learn patterns",
    "this is a simple example"
]

# -------------------------
# 2. Tokenization
# -------------------------
# Build vocabulary
words = set()
for sent in sentences:
    words.update(sent.split())

word2idx = {w: i+1 for i, w in enumerate(sorted(words))}
word2idx["<pad>"] = 0
idx2word = {i: w for w, i in word2idx.items()}

vocab_size = len(word2idx)

# Convert sentences to token indices
max_len = max(len(s.split()) for s in sentences)

tokenized = []
for sent in sentences:
```

```
    tokens = [word2idx[w] for w in sent.split()]
    # pad
    tokens += [0] * (max_len - len(tokens))
    tokenized.append(tokens)

input_tensor = torch.tensor(tokenized)

print("Input Tokens:\n", input_tensor)
print("Shape:", input_tensor.shape)

# -------------------------
# 3. Positional Encoding
# -------------------------
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=50):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len).unsqueeze(1)

        div_term = torch.exp(torch.arange(0, d_model, 2) * (-
np.log(10000.0) / d_model))

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)

        self.pe = pe.unsqueeze(0)

    def forward(self, x):
        return x + self.pe[:, :x.size(1)]

# -------------------------
# 4. Transformer Encoder Block
# -------------------------
class MiniTransformerEncoder(nn.Module):
    def __init__(self, vocab_size, d_model=32, num_heads=2, d_ff=64):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model)
        self.pos_encoding = PositionalEncoding(d_model)

        self.attention = nn.MultiheadAttention(embed_dim=d_model,
num_heads=num_heads, batch_first=True)

        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
```

```python
        self.ff = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.ReLU(),
            nn.Linear(d_ff, d_model)
        )

    def forward(self, x):
        # Embedding + Positional Encoding
        emb = self.embedding(x)
        emb = self.pos_encoding(emb)

        # Self-attention
        attn_output, attn_weights = self.attention(emb, emb, emb)

        # Add & Norm
        x = self.norm1(emb + attn_output)

        # Feed-forward
        ff_output = self.ff(x)

        # Add & Norm
        out = self.norm2(x + ff_output)

        return out, attn_weights

# -------------------------
# 5. Run Model
# -------------------------
model = MiniTransformerEncoder(vocab_size)

output, attn_weights = model(input_tensor)

print("\nFinal Contextual Embeddings Shape:", output.shape)
print("\nSample Embedding for first sentence:\n", output[0])

# -------------------------
# 6. Attention Heatmap
# -------------------------
# Visualize attention of first sentence
attn = attn_weights[0].detach().numpy()

plt.imshow(attn, cmap='viridis')
plt.colorbar()
plt.title("Attention Heatmap (Sentence 1)")
```

```
words_list = sentences[0].split()
plt.xticks(range(len(words_list)), words_list, rotation=45)
plt.yticks(range(len(words_list)), words_list)

plt.show()
```

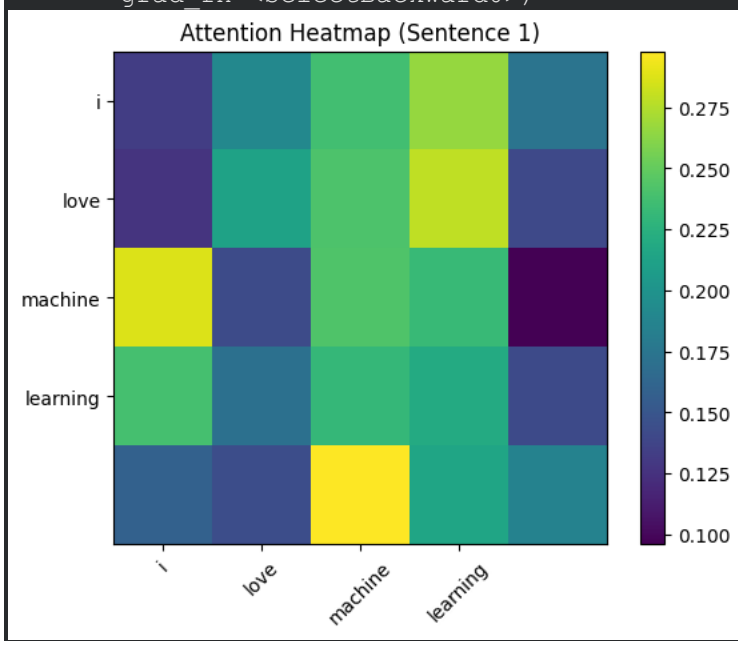
Output:

```
Input Tokens:
 tensor([[13, 19, 20, 18,  0],
        [36,  3, 28, 22,  0],
        [ 6, 18, 15,  2,  0],
        [23, 16, 29, 15, 11],
        [ 4, 12, 22, 10,  0],
        [30, 21, 35,  7,  0],
        [13,  8, 18, 26, 33],
        [ 5, 31, 15, 14,  0],
        [25, 24, 17, 27,  0],
        [34, 15,  1, 32,  9]])
Shape: torch.Size([10, 5])

Final Contextual Embeddings Shape: torch.Size([10, 5, 32])

Sample Embedding for first sentence:
 tensor([[ 0.3868, -0.4623, -0.8583,  1.2858,  1.2400, -0.0036, -1.5847,
1.7211,
         -0.1155,  0.3886, -1.9526,  1.0675,  0.9535, -0.2833,  0.0955,
1.6584,
          0.3598, -0.2237, -0.5702, -0.9992, -1.8268, -0.8043,  1.2375,
1.0487,
          1.0936, -0.6841, -0.8609,  0.1001, -0.5286,  0.5445, -1.5016,
0.0785],
        [ 1.2159, -0.4647, -0.4379, -0.9480, -0.3755,  1.6148, -0.3960, -
1.0746,
         -0.2175,  0.4312, -1.7452,  1.7336, -0.7720, -1.4337,  0.5779,
0.9565,
          0.2704, -0.9084, -0.7907, -0.5346, -0.7921,  0.7325,  0.4968,
1.0904,
         -1.1550,  1.5008, -1.4450,  1.5760,  0.4290, -0.7510,  0.3904,
1.2255],
        [ 0.1148, -1.1008,  0.8657,  0.7095,  0.6982,  0.4019, -0.9795,
0.4658,
          0.9057,  0.2246,  1.2264,  0.3479, -0.2854,  2.0566, -1.0734,
1.0393,
         -1.0022, -1.1713, -1.0555,  0.1457, -1.1954, -0.1807,  0.1431,
0.0948,
         -2.2716,  0.4111, -1.4013, -0.7840, -0.3288,  1.5255, -0.3355,
1.7888],
        [-2.0227, -2.4267, -0.3408,  0.9029,  0.1961,  0.7334, -0.1920,
0.4030,
          0.7216,  0.5474, -0.7005, -1.3609, -1.4958, -0.0181,  1.4207,
1.0608,
          0.1213,  0.6542,  0.8903,  2.1148,  0.2537, -0.1842,  0.4945, -
0.9455,
         -0.5642, -0.5333, -0.6154, -1.1240, -0.1696, -0.0265,  0.8608,
1.3447],
```

```
        [ 0.4877,  0.0341,  1.1656, -1.9470,  0.4742,  0.7760, -0.2934,
1.3569,
          0.4118,  0.1993, -1.6642,  2.1188, -0.4337,  0.3241, -0.4429,
0.9917,
         -1.3000,  2.0059, -0.0978,  0.5833, -0.4567,  0.9164, -0.1454, -
0.3280,
         -0.7123, -1.3691, -1.5084,  0.9360,  0.0317, -1.3623, -0.3408, -
0.4113]],
       grad_fn=<SelectBackward0>)
```



Q3. Implement Scaled Dot-Product Attention

Goal: Implement the attention function from your slides:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Requirements:

- Write a function in PyTorch or TensorFlow to compute attention.
- Test it using random Q, K, V inputs.
- Print:
  - Attention weight matrix
  - Output vectors
  - Softmax stability check (before and after scaling)

Code:

```
# scaled_dot_product_attention.py
```

```python
import torch
import torch.nn.functional as F
import numpy as np

# -------------------------
# 1. Scaled Dot-Product Attention Function
# -------------------------
def scaled_dot_product_attention(Q, K, V):
    d_k = Q.size(-1)  # dimension of keys

    # Step 1: Compute raw attention scores
    scores = torch.matmul(Q, K.transpose(-2, -1))  # QK^T

    # Step 2: Scale
    scaled_scores = scores / torch.sqrt(torch.tensor(d_k,
dtype=torch.float32))

    # Step 3: Softmax
    attn_weights = F.softmax(scaled_scores, dim=-1)

    # Step 4: Multiply with V
    output = torch.matmul(attn_weights, V)

    return scores, scaled_scores, attn_weights, output


# -------------------------
# 2. Generate Random Q, K, V
# -------------------------
batch_size = 1
seq_len = 4
d_k = 8

torch.manual_seed(42)

Q = torch.randn(batch_size, seq_len, d_k)
K = torch.randn(batch_size, seq_len, d_k)
V = torch.randn(batch_size, seq_len, d_k)

print("Q:\n", Q)
print("\nK:\n", K)
print("\nV:\n", V)
```

```python
# -------------------------
# 3. Compute Attention
# -------------------------
scores, scaled_scores, attn_weights, output =
scaled_dot_product_attention(Q, K, V)


# -------------------------
# 4. Without Scaling (for comparison)
# -------------------------
raw_scores = torch.matmul(Q, K.transpose(-2, -1))
attn_no_scale = F.softmax(raw_scores, dim=-1)


# -------------------------
# 5. Print Results
# -------------------------
np.set_printoptions(precision=3, suppress=True)

print("\n==============================")
print("Raw Attention Scores (QK^T):")
print(scores.detach().numpy())

print("\nScaled Scores (QK^T / sqrt(d_k)):")
print(scaled_scores.detach().numpy())

print("\n==============================")
print("Softmax WITHOUT scaling:")
print(attn_no_scale.detach().numpy())

print("\nSoftmax WITH scaling:")
print(attn_weights.detach().numpy())

print("\n==============================")
print("Attention Output (Weighted Sum of V):")
print(output.detach().numpy())


# -------------------------
# 6. Stability Check
# -------------------------
print("\n==============================")
print("Stability Check:")

print("\nMax value BEFORE scaling:", torch.max(scores).item())
```

```
print("Max value AFTER scaling:", torch.max(scaled_scores).item())

print("\nSoftmax distribution WITHOUT scaling:")
print(attn_no_scale[0][0])

print("\nSoftmax distribution WITH scaling:")
print(attn_weights[0][0])
```

Output:

```
Q:
 tensor([[[ 1.9269,  1.4873,  0.9007, -2.1055,  0.6784, -1.2345, -0.0431,
          -1.6047],
         [-0.7521,  1.6487, -0.3925, -1.4036, -0.7279, -0.5594, -0.7688,
           0.7624],
         [ 1.6423, -0.1596, -0.4974,  0.4396, -0.7581,  1.0783,  0.8008,
           1.6806],
         [ 1.2791,  1.2964,  0.6105,  1.3347, -0.2316,  0.0418, -0.2516,
           0.8599]]])

K:
 tensor([[[-1.3847, -0.8712, -0.2234,  1.7174,  0.3189, -0.4245,  0.3057,
          -0.7746],
         [-1.5576,  0.9956, -0.8798, -0.6011, -1.2742,  2.1228, -1.2347,
          -0.4879],
         [-0.9138, -0.6581,  0.0780,  0.5258, -0.4880,  1.1914, -0.8140,
          -0.7360],
         [-1.4032,  0.0360, -0.0635,  0.6756, -0.0978,  1.8446, -1.1845,
           1.3835]]])

V:
 tensor([[[ 1.4451,  0.8564,  2.2181,  0.5232,  0.3466, -0.1973, -1.0546,
           1.2780],
         [-0.1722,  0.5238,  0.0566,  0.4263,  0.5750, -0.6417, -2.2064,
          -0.7508],
         [ 0.0109, -0.3387, -1.3407, -0.5854,  0.5362,  0.5246,  1.1412,
           0.0516],
         [ 0.7440, -0.4816, -1.0495,  0.6039, -1.7223, -0.8278,  1.3347,
           0.4835]]])

================================
Raw Attention Scores (QK^T):
[[[-5.811 -3.696 -4.362 -8.643]
  [-3.538  4.319 -1.413  1.196]
  [-3.025 -1.097 -1.438  1.458]
  [-1.579 -1.766 -1.538  0.702]]]

Scaled Scores (QK^T / sqrt(d_k)):
[[[-2.054 -1.307 -1.542 -3.056]
  [-1.251  1.527 -0.5    0.423]
  [-1.07  -0.388 -0.508  0.516]
  [-0.558 -0.624 -0.544  0.248]]]

================================
```

```
Softmax WITHOUT scaling:
[[[0.074 0.609 0.313 0.004]
  [0.    0.955 0.003 0.042]
  [0.01  0.068 0.048 0.874]
  [0.079 0.066 0.082 0.773]]]

Softmax WITH scaling:
[[[0.194 0.41  0.324 0.071]
  [0.041 0.656 0.086 0.217]
  [0.104 0.206 0.182 0.508]
  [0.193 0.18  0.195 0.432]]]

==============================
Attention Output (Weighted Sum of V):
[[[ 0.267  0.237 -0.055  0.13   0.354 -0.191 -0.645 -0.009]
  [ 0.109  0.244 -0.216  0.381  0.063 -0.563 -1.101 -0.331]
  [ 0.495 -0.109 -0.535  0.342 -0.622 -0.477  0.322  0.233]
  [ 0.571 -0.015 -0.277  0.324 -0.468 -0.408  0.198  0.33 ]]]

==============================
Stability Check:

Max value BEFORE scaling: 4.319219589233398
Max value AFTER scaling: 1.5270748138427734

Softmax distribution WITHOUT scaling:
tensor([0.0735, 0.6092, 0.3130, 0.0043])

Softmax distribution WITH scaling:
tensor([0.1942, 0.4102, 0.3242, 0.0714])
```